

T.C.
NECMETTİN ERBAKAN ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BİLGİSAYAR MÜHENDİSLİĞİ UYGULAMA TASARIMI
PROJE FİNAL DENEY RAPORU

Kısıtlı IoT Ortamlarında MQTT-SN ve CoAP Protokollerinin
FIT IoT-LAB Test Yatağı Üzerinde Karşılaştırmalı Başarım
Değerlendirmesi:
Star, Doğrusal ve Ağaç Topolojisi Senaryoları

Hazırlayan: Hakan DÖNMEZ
Öğrenci No: 20010011038

Danışman: [ALPEREN EROĞLU]

Tarih: Mayıs 2026

ÖZET

Bu çalışmada, Nesnelerin İnterneti (IoT) ekosisteminde yaygın biçimde kullanılan iki uygulama katmanı iletişim protokolü olan MQTT-SN (Message Queuing Telemetry Transport for Sensor Networks) ve CoAP (Constrained Application Protocol), gerçek bir donanım test yatağı üzerinde deneysel olarak karşılaştırılmıştır. Deneyler, Fransa'daki FIT IoT-LAB altyapısı bünyesindeki Saclay test alanında konuşlandırılan A8-M3 düğümleri ve RIOT OS işletim sistemi kullanılarak yürütülmüştür. Protokoller; yıldız (star), doğrusal (linear) ve ağaç (tree) topolojileri altında 802.15.4/6LoWPAN kablosuz bağlantı standardı ile çalıştırılmış; uçtan uca gecikme (RTT), paket iletim oranı (PDR) ve verimlilik (throughput) metrikleri ölçülmüştür. Elde edilen bulgular, MQTT-SN protokolünün ağaç topolojisinde 15,95 ms ortalama RTT ve %100 PDR ile son derece kararlı bir başarımla sergilediğini ortaya koymuştur. CoAP protokolü ise yıldız topolojisinde %100 PDR elde ederek MQTT-SN'e kıyasla daha az topoloji bağımlılığı göstermiştir. Her iki protokolde de topolojik mesafe arttıkça gecikme ve paket kayıplarının belirgin biçimde arttığı gözlemlenmiştir. Bu sonuçlar, protokol seçiminin topoloji, ağ yoğunluğu ve uygulama gereksinimleriyle birlikte değerlendirilmesi gerektiğine işaret etmektedir.

Anahtar Kelimeler: IoT, MQTT-SN, CoAP, RIOT OS, FIT IoT-LAB, 6LoWPAN, RTT, PDR, Başarımla Değerlendirmesi

İÇİNDEKİLER

1. GİRİŞ	4
2. LİTERATÜR TARAMASI	5
3. KULLANILAN DONANIM VE YAZILIM ALTYAPISI	6
3.1. FIT IoT-LAB Test Yatağı	6
3.2. A8-M3 Düğüm Mimarisi	6
3.3. RIOT OS İşletim Sistemi	6
3.4. İletişim Protokolleri	7
4. DENEY METODOLOJİSİ	8
4.1. Topoloji Tasarımları	8
4.2. Ölçüm Metrikleri	9
4.3. Yazılım Mimarisi ve Firmware	9
4.4. Deney Yaşam Döngüsü	10
5. DENEYSEL BULGULAR VE ANALİZ	11
5.1. Uçtan Uca Gecikme (RTT)	11
5.2. Paket İletim Oranı (PDR)	12
5.3. Wireshark Trafik Analizi	12
5.4. Karşılaştırmalı Değerlendirme	13
6. TARTIŞMA	14
7. HAFTALIK İLERLEME RAPORU	15
8. SONUÇ VE ÖNERİLER	20
KAYNAKÇA	21

1. GİRİŞ

Nesnelerin İnterneti (IoT) paradigması, fiziksel dünyanın dijital sistemlerle entegrasyonunu sağlayan milyarlarca bağlı cihazın birbirleriyle iletişim kurduğu bir ekosistemi tanımlamaktadır [1]. Bu ekosistemde çalışan düğümler tipik olarak kısıtlı işlem kapasitesi, sınırlı bellek ve düşük enerji bütçesiyle karakterize edilmektedir. Söz konusu donanımsal kısıtlar, geleneksel İnternet protokollerinin (HTTP, AMQP gibi) doğrudan uygulanmasını güçleştirmekte; bu nedenle IoT dünyasına özgü hafif iletişim protokollerine olan ihtiyacı doğurmaktadır [2].

Bu bağlamda öne çıkan iki temel protokol MQTT-SN ve CoAP'tir. MQTT-SN (MQTT for Sensor Networks), yayımcı-abone (publish-subscribe) mimarisine dayanan MQTT protokolünün 802.15.4 gibi kayıplı ağlar için optimize edilmiş türevidir [3]. CoAP (Constrained Application Protocol) ise REST mimarisini kısıtlı cihazlara taşıyan, UDP tabanlı bir istek-yanıt protokolüdür [4]. Her iki protokol de IETF standartları çerçevesinde tanımlanmış olmakla birlikte, farklı mesajlaşma paradigmaları ve ağ gereksinimleri nedeniyle birbirinden ayrılmaktadır.

Bu çalışmanın özgün katkısı, söz konusu iki protokolün gerçek donanım üzerinde —simülasyon ortamı değil— üç farklı ağ topolojisinde eş zamanlı ve tekrarlanabilir biçimde karşılaştırılmasıdır. FIT IoT-LAB gibi büyük ölçekli bir açık test yatağının kullanılması, sonuçların akademik tekrarlanabilirlik (reproducibility) standardını karşılamasına olanak tanımaktadır. Bu boyutuyla çalışma, literatürdeki mevcut simülasyon tabanlı karşılaştırma çalışmalarını gerçek ölçüm verileriyle destekleyerek bilimsel birikime katkıda bulunmayı hedeflemektedir.

Raporun geri kalanı şu şekilde yapılandırılmıştır: Bölüm 2'de ilgili literatür incelenmekte, Bölüm 3'te donanım ve yazılım altyapısı tanıtılmakta, Bölüm 4'te deney metodolojisi açıklanmakta, Bölüm 5'te bulgular sunulmakta, Bölüm 6'da bulgular tartışılmakta, Bölüm 7'de haftalık ilerleme raporları yer almakta ve Bölüm 8'de sonuç ve öneriler verilmektedir.

2. LİTERATÜR TARAMASI

Kısıtlı IoT protokollerinin karşılaştırılmasına yönelik çalışmalar son on yılda hız kazanmıştır. Palmese ve arkadaşları [5], yayımcı-abone ortamlarında MQTT-SN ve CoAP'ı gecikme ve mesaj büyüklüğü açısından değerlendirmiş; CoAP'ın düşük yük trafiğinde daha az ek yük (overhead) ürettiğini, MQTT-SN'in ise yoğun yayın senaryolarında avantaj sağladığını ortaya koymuştur.

Colitti ve arkadaşları [6], CoAP'ı HTTP ile karşılaştırmış ve CoAP'ın kısıtlı düğümlerde yüzde seksen oranında daha az bant genişliği tükettiğini göstermiştir. MQTT-SN spesifikasyonu Stanford–Clark ve Nipper [3] tarafından tanımlanmış olup protokolün düşük güçlü sensör ağlarına uyarlanmasına odaklanmaktadır. Villaverde ve arkadaşları ise 6LoWPAN ortamında CoAP'ın güvenilirliğini incelemiş; CON (Confirmable) mesaj tipinin gecikme maliyetine karşın iletim güvenilirliğini önemli ölçüde artırdığını bulmuştur [7].

FIT IoT-LAB altyapısını inceleyen Adjih ve arkadaşları [8], test yatağının tekrarlanabilir deneyler için sağladığı avantajları vurgulamıştır. Mevcut literatürde gerçek donanım üzerinde her iki protokolün üç farklı topolojide eş zamanlı karşılaştırıldığı bir çalışmaya rastlanmamaktadır. Bu boşluk, söz konusu çalışmanın temel motivasyonunu oluşturmaktadır.

3. KULLANILAN DONANIM VE YAZILIM ALTYAPISI

3.1. FIT IoT-LAB Test Yatağı

FIT IoT-LAB (Future Internet Testbed with IoT), Fransa'da birden fazla sitede konuşlandırılmış, akademik ve araştırma amaçlı büyük ölçekli bir açık IoT test altyapısıdır [8]. Bu çalışmada ağırlıklı olarak Saclay sitesi kullanılmış; deneyler SSH üzerinden uzaktan erişim (donmez@sacclay.iot-lab.info) ile yönetilmiştir. Test yatağı; deney rezervasyonu, düğüm flaşlama ve izleme için iotlab-experiment, iotlab-ssh ve iotlab-auth komut satırı araçlarını sunmaktadır.

3.2. A8-M3 Düğüm Mimarisi

Tablo 1. A8-M3 Düğümü Temel Donanım Özellikleri

Bileşen	Özellik
İşlemci (Uygulama)	ARM Cortex-A8, 600 MHz
İşlemci (M3 – Radyo)	STM32 ARM Cortex-M3, 72 MHz
RAM (A8)	256 MB DDR
Flash (M3)	256 KB
Radyo	AT86RF231 – IEEE 802.15.4 (2,4 GHz)
İşletim Sistemi	RIOT OS (M3 tarafı), Linux (A8 tarafı)

A8-M3 düğümü çift işlemcili bir mimariye sahiptir: Linux çalıştıran ARM Cortex-A8 işlemcisi ağ geçidi (border router) rolünü üstlenirken, RIOT OS çalıştıran STM32 Cortex-M3 işlemcisi 802.15.4 radyo üzerinden sensör ağı iletişimini yönetmektedir. Bu iki işlemci arasındaki iletişim, 500.000 baud hızındaki UART/EthoS köprüsü aracılığıyla gerçekleştirilmektedir.

3.3. RIOT OS İşletim Sistemi

RIOT OS, kısıtlı gömülü sistemler için tasarlanmış, gerçek zamanlı yetenekler sunan açık kaynaklı bir mikrodenetleyici işletim sistemidir [9]. Modüler mimarisi sayesinde yalnızca ihtiyaç duyulan ağ yığını bileşenleri (6LoWPAN, RPL, emcute, gcoap) derlemeye dahil edilebilmektedir. Bu çalışmada kullanılan temel RIOT OS modülleri şunlardır: gnrc_border_router (ağ geçidi firmware'i), emcute (MQTT-SN istemci kitaplığı) ve gcoap (CoAP istemci/sunucu kitaplığı).

3.4. İletişim Protokolleri

3.4.1. MQTT-SN (emcute)

MQTT-SN, MQTT'nin 802.15.4 gibi kayıplı bağlantılara uyarlanmış biçimidir. Yayımcı-abone mimarisinde çalışır; mesajlar bir broker (bu çalışmada Mosquitto RSMB) üzerinden yönlendirilir. RIOT OS bünyesindeki emcute kütüphanesi, MQTT-SN istemci işlevlerini (connect, subscribe, publish) sağlamaktadır. QoS 0 (en iyi çaba, onaysız iletim) modu kullanılmıştır.

3.4.2. CoAP (gcoap)

CoAP, UDP tabanlı REST semantiğini kısıtlı cihazlara taşıyan bir protokoldür. RIOT OS'un gcoap modülü, asenkron istek-yanıt mekanizmasını desteklemektedir. Bu çalışmada Confirmable (CON) POST iletileri kullanılmış; sunucu düğüm /metrics/rtt kaynağında gelen veriyi echo olarak geri göndermiştir.

Tablo 2. MQTT-SN ve CoAP Protokollerinin Temel Özellik Karşılaştırması

Özellik	MQTT-SN	CoAP
---------	---------	------

Mesajlaşma Modeli	Yayımcı-Abone	İstek-Yanıt
Taşıma Katmanı	UDP	UDP
Broker Gereksinimi	Evet (RSMB)	Hayır
QoS Seviyeleri	0, 1, 2, -1	NON, CON (ACK)
Başlık Boyutu	2 bayt (min)	4 bayt (min)
IETF Standardı	OASIS / taslak	RFC 7252
RIOT OS Modülü	emcute	gcoap

4. DENEY METODOLOJİSİ

4.1. Topoloji Tasarımları

Protokollerin ağ topolojisine duyarlılığını incelemek amacıyla üç farklı topoloji kurgulanmıştır. Çok atlamalı (multi-hop) topolojiler, düğümlerin iletim gücü (TX power) parametresi kısıtlanarak elde edilmiştir. Her topolojide 15 A8-M3 düğüm görev almıştır.

- Yıldız (Star) Topolojisi: Tüm istemci düğümlerin doğrudan sınır yönlendiriciye (border router) bağlandığı tek atlamalı mimari. Topoloji, en düşük gecikme potansiyelini sunar ancak yoğun trafikte tıkanma riski taşır.
- Doğrusal (Linear) Topoloji: Düğümlerin seri biçimde dizildiği zincir mimari. Her paket hedefe ulaşmak için birden fazla yönlendirme atlaması gerektirir; gecikme her atlama ile artmaktadır.
- Ağaç (Tree) Topolojisi: RPL protokolünün doğal yönlendirme hiyerarşisini yansıtan; kök (root) ile yapraklar arasında birden fazla dal bulunan hiyerarşik mimari. Gerçek dağıtık IoT ağlarını en yakın şekilde temsil eden topolojidir.

4.2. Ölçüm Metrikleri

Deneyssel başarımların değerlendirilmesinde aşağıdaki metrikler kullanılmıştır:

- Uçtan Uca Gecikme / RTT (Round-Trip Time): Yük verisi içine yerleştirilen zaman damgası (payload-embedded timestamp) yöntemiyle ölçülmüştür. Gönderme ve alım anları `ztimer_now()` çağrısıyla ms hassasiyetinde kaydedilmiş; fark RTT olarak hesaplanmıştır.
- Paket İletim Oranı / PDR (Packet Delivery Ratio): Başarıyla alınan paket sayısının gönderilen toplam paket sayısına oranı olarak yüzde cinsinden ifade edilmiştir.
- Verimlilik (Throughput): Wireshark yakalama dosyalarından (PCAP) elde edilen paket hızı (pps) ve ortalama paket boyutu kullanılarak hesaplanmıştır.

Her deney çalıştırmasında 200 mesaj gönderilmiş; ağın RPL yakınsama sürecini tamamlaması için 5 saniyelik ısınma (warm-up) süresi uygulanmıştır.

4.3. Yazılım Mimarisi ve Firmware

Her iki protokol için ayrı firmware geliştirilmiştir. MQTT-SN firmware'i (emcute_mqtttns), emcute kütüphanesini kullanarak brokera bağlanmakta, metrics/rtt konusuna abone olmakta ve 200 mesaj yayımlamaktadır. Her yayımlanan mesaj, sıra numarası ve ms cinsinden zaman damgasını içermektedir. Broker üzerinden geri dönen echo, abone callback fonksiyonunda yakalanmakta ve RTT hesaplanmaktadır.

CoAP firmware'i (gcoap), rol tabanlı bir mimariye sahiptir: bir düğüm "server" rolüyle /metrics/rtt kaynağını sunmakta; diğer düğümler "client" rolüyle CON POST istekleri göndermektedir. Sunucu,

gelen yükü değiştirmeksizin (echo) 2.05 Content yanıtıyla geri döndürmekte; istemci yanıt alındığında RTT'yi hesaplamaktadır. Her iki firmware'de de ölçüm verileri CSV formatında seri porta yazdırılmaktadır.

4.4. Deney Yaşam Döngüsü

Deney süreci aşağıdaki aşamalardan oluşmaktadır:

Tablo 3. Deney Yaşam Döngüsü Adımları

Adım	İşlem	Araç/Komut
1	Deney rezervasyonu	iotlab-experiment submit -n riot_a8 -d 120 -l 15,archi=a8:at86rf231+site=sacalay
2	Border router firmware derleme	make ETHOS_BAUDRATE=500000 DEFAULT_CHANNEL=26 BOARD=iotlab-a8-m3 -C examples/gnrc_border_router
3	Protokol firmware derleme	make DEFAULT_CHANNEL=26 BOARD=iotlab-a8-m3 -C firmware/mqtt_node
4	A8 Linux tarafı flaşlama	iotlab_flash ~/A8/gnrc_border_router.elf
5	M3 radyo tarafı flaşlama	iotlab-ssh flash-m3 emcute_mqttsn.elf -l sacalay,a8,103
6	Seri port izleme & veri toplama	miniterm.py /dev/ttyA8_M3 500000 -e

Başarılı olarak tamamlanan deney kimlik numaraları şunlardır: 434217 (MQTT-SN, ağaç topolojisi, 10 düğüm, 120 dakika), 432809 ve 432999 (ön denemeler), 434071 ve 434104 (CoAP denemeleri).

5. DENEYSEL BULGULAR VE ANALİZ

5.1. Uçtan Uca Gecikme (RTT)

Tablo 4. Topoloji Başına RTT İstatistikleri (ms cinsinden)

Senaryo	Protokol	Ort. RTT	Med. RTT	Min RTT	Maks RTT	Std Sapma
Star	MQTT-SN	1983,90	2599,82	14,80	4358,84	1850,32
Star	CoAP	17,67	17,40	15,44	23,86	1,40
Linear	MQTT-SN	362,64	697,99	27,28	697,99	335,35
Linear	CoAP	17,97	17,70	16,57	19,72	1,00
Tree	MQTT-SN	15,95	15,59	14,25	19,62	1,34
Tree	CoAP	205,88	20,54	15,96	4796,87	884,49

RTT ölçüm sonuçları incelendiğinde en dikkat çekici bulgu, MQTT-SN'in ağaç topolojisinde son derece kararlı ve düşük gecikme (ort. 15,95 ms, std. 1,34 ms) sergilemesidir. Bu değer, CoAP'ın aynı topolojideki ortalamasının (205,88 ms) yaklaşık on üç katı daha iyi olduğunu göstermektedir. CoAP'ın ağaç topolojisindeki yüksek ortalama değeri, CoAP yeniden iletim (retransmission) mekanizmasından kaynaklanan aykırı değerler (max 4796,87 ms) nedeniyle çarpıtılmıştır; medyan değerin (20,54 ms) düşük olması bu yorumu desteklemektedir.

Öte yandan CoAP, yıldız ve doğrusal topolojilerde çok daha kararlı sonuçlar üretmiş; RTT ortalaması sırasıyla 17,67 ms ve 17,97 ms olarak ölçülmüştür. MQTT-SN'in yıldız topolojisindeki yüksek gecikme (1983,90 ms), broker'ın tek nokta olarak yoğun istemci yüküyle başa çıkamamasından kaynaklanmaktadır.

5.2. Paket İletim Oranı (PDR)

Tablo 5. Topoloji Başına PDR Değerleri (%)

Topoloji	MQTT-SN PDR (%)	CoAP PDR (%)
Star	38,46	100,00
Linear	100,00 *	61,90
Tree	100,00	84,85

* MQTT-SN doğrusal topoloji PDR değeri yalnızca 2 başarılı çift üzerinden hesaplanmıştır; bu nedenle istatistiksel anlamlılığı sınırlıdır.

PDR sonuçları, CoAP'ın yıldız topolojisinde %100 güvenilir iletim sağlarken MQTT-SN'nin yalnızca %38,46 oranında başarılı olduğunu göstermektedir. Bu fark, iki protokolün mesajlaşma mimarilerinden kaynaklanmaktadır: CoAP'ın CON mesajları, otomatik yeniden iletim mekanizmasıyla paketi hedefe ulaştırmayı garantilerken, MQTT-SN QoS 0 iletimi herhangi bir onay mekanizması içermemektedir.

Öte yandan MQTT-SN, ağaç topolojisinde %100 PDR elde etmiştir. Bu sonuç, RPL yönlendirme protokolünün ağaç yapısında MQTT-SN trafiğini verimli biçimde yönettiğine işaret etmektedir. CoAP'ın doğrusal topolojideki %61,9 PDR değeri ise zincir mimarisinde çok atlamalı yönlendirmenin paket kayıplarını nasıl artırdığını açıkça ortaya koymaktadır.

5.3. Wireshark Trafik Analizi

Deneyler süresince sınır yönlendirici üzerinde Wireshark/tcpdump ile PCAP dosyaları yakalanmıştır. Tablo 6, her senaryo için temel PCAP istatistiklerini özetlemektedir.

Tablo 6. PCAP Dosyalarından Elde Edilen Trafik İstatistikleri

Senaryo	Toplam Paket	Süre (s)	Ort. Paket Boyutu (B)	Throughput (pps)
MQTT-SN Star	680	2325,66	85,3	0,29
MQTT-SN Linear	726	799,09	86,3	0,91
MQTT-SN Tree	562	144,54	104,0	3,89
CoAP Star	79	142,44	97,6	0,55
CoAP Linear	147	635,10	100,5	0,23
CoAP Tree	163	535,16	101,7	0,30

MQTT-SN ağaç topolojisi, en yüksek throughput değerini (3,89 pps) ve en kısa deney süresini (144,54 s) ortaya koymuştur. Bu durum, MQTT-SN'in ağaç hiyerarşisini verimli biçimde kullandığını ve RPL yönlendirmesiyle senkronize çalıştığını göstermektedir. CoAP'ın genel olarak daha büyük ortalama paket boyutu üretmesi (97,6–101,7 B), gcoap çerçevesinin eklediği seçenekler (Option) ve Token alanlarından kaynaklanmaktadır.

5.4. Karşılaştırmalı Değerlendirme

Şekil 1'de RTT karşılaştırma grafiği, Şekil 2'de ise PDR karşılaştırma grafiği sunulmuştur. Her iki grafikte de sonuçların topolojiye olan güçlü bağımlılığı açıkça görülmektedir. Tüm senaryolar bir arada değerlendirildiğinde ne MQTT-SN ne de CoAP'ın her koşulda üstün olmadığı; başarımın topoloji ve uygulama gereksinimlerine göre belirgin biçimde farklılaştığı anlaşılmaktadır.

Şekil 1. Topoloji bazında RTT karşılaştırması (results/comparison_plot.png)

Şekil 2. Derinlemesine başarımlar karşılaştırması (results/deep_comparison.png)

6. TARTIřMA

Elde edilen bulgular, protokol seiminin yalnızca teknik zelliklerle deėil, daėıtım ortamının topolojik yapısıyla da yakından iliřkili olduėunu ortaya koymaktadır. Bu bulgu, mevcut literatürdeki simölasyon tabanlı alıřmaların öngörülerine kısmen aykırıdır: Simölasyon alıřmalarının büyük bölümü CoAP'ı yayımcı-abone senaryolarında dezavantajlı konumda gösterirken, gerek donanım üzerinde elde edilen sonuçlar CoAP'ın yıldız topolojisinde rakipsiz PDR başarımı sergilediėini ortaya koymuřtur [5].

MQTT-SN'in aėaç topolojisindeki üstün RTT başarımı, RPL yönlendirme protokolünün MQTT-SN'in broker merkezli yapısıyla daha iyi örtüřtüėünü düşündürmektedir. RPL, aėaç topolojisini doėal hiyerarřisiyle desteklediėinden, mesajların broker'a ulaşması için gereken yönlendirme süresi minimuma inmektedir. Bu bulgu, gerek akıllı bina veya endüstriyel izleme uygulamaları için protokol seiminde topoloji tasarımının kritik bir parametre olarak deėerlendirilmesi gerektiėini vurgulamaktadır.

alıřmanın sınırlılıkları arasında řunlar sayılabilir: (1) MQTT-SN doėrusal topoloji PDR'si yalnızca 2 başarılı paket ifti üzerinden hesaplanmıřtır; daha güvenilir istatistik için deney tekrarlanmalıdır. (2) CoAP aėaç topolojisindeki yüksek RTT ortalaması, küçük sayıda aykırı deėere aşırı duyarlı olabilir. (3) Deneyler belirli bir radyo kanalında (kanal 26) yürütölmüş; farklı kanallarda girişim düzeyi farklılaşabilir. Gelecek alıřmalarda QoS 1/2 modu, řifreleme (DTLS/TLS) etkisi ve enerji tüketimi metrikleri araştırılmalıdır.

7. HAFTALIK İLERLEME RAPORU

Aşağıda 14 haftalık çalışma süreci boyunca gerçekleştirilen faaliyetler, karşılaşılan teknik sorunlar ve uygulanan çözümler ayrıntılı biçimde aktarılmaktadır.

Hafta 1 (Şubat 2026, 1. Hafta): Proje Kurulumu ve Literatür Taraması

- Bu hafta projenin kapsamı ve motivasyonu netleştirilmiştir. FIT IoT-LAB platformu incelenmiş; kullanıcı hesabı oluşturulmuş ve komut satırı araçları (iotlab-auth, iotlab-experiment) kurulmuştur. Karşılaştırılacak protokollere (MQTT-SN, CoAP) ilişkin temel akademik makaleler taranmıştır [3,4,5,8].
- Tarama yapılan başlıca çalışmalar: RFC 7252 (CoAP spesifikasyonu), Stanford-Clark & Nipper MQTT-SN taslağı, Palmese et al. (CoAP vs. MQTT-SN karşılaştırması), Adjih et al. (FIT IoT-LAB tanıtım makalesi).
- Proje GitHub deposu oluşturulmuş; klasör yapısı (firmware/, scripts/, results/, deneyler/, makaleler/) düzenlenmiştir.
- Başarı Ölçütü: Literatür taraması tamamlandı, proje formu dolduruldu, GitHub deposu oluşturuldu.

Hafta 2 (Şubat 2026, 2. Hafta): RIOT OS Ortamı Kurulumu

- RIOT OS kaynak kodu yerel makinede derlendi. BOARD=iotlab-a8-m3 hedefi için gerekli araç zinciri (ARM GCC) kuruldu. İlk olarak örnekler/ dizinindeki gnrc_networking örneği derlenerek donanım hedefi tanındı.
- Teknik Sorun: Araç zinciri sürüm uyumsuzluğu nedeniyle derleme hataları alındı (arm-none-eabi-gcc 10.x yerine 12.x gerekiyordu). Çözüm: apt-get ile güncel araç zinciri kuruldu.
- FIT IoT-LAB üzerinde ilk deney rezervasyonu yapılmış (Strasbourg, 4 düğüm, 60 dakika – Deney #433089); gnrc_networking örneği flaşlanarak ağ bağlantısı doğrulandı.
- Başarı Ölçütü: RIOT OS derleme ortamı çalışır duruma getirildi; ilk deney rezervasyonu gerçekleştirildi.

Hafta 3 (Şubat 2026, 3. Hafta): Border Router Yapılandırması

- gnrc_border_router firmware'i derlendi ve A8 düğümünün Linux tarafına flaşlandı (iotlab_flash ~/A8/gnrc_border_router.elf). EthoS köprüsü üzerinden M3 radyo tarafı ile IPv6 bağlantısı kuruldu.
- IPv6 adresi başarıyla alındı (2001:660:3207:400::65/64). ping -6 komutuyla border router ile M3 düğümleri arasındaki bağlantı test edildi.
- Teknik Sorun: EthoS baud rate (500.000) uyumsuzluğu nedeniyle seri iletişim kurulamadı. Çözüm: make komutuna ETHOS_BAUDRATE=500000 parametresi eklendi.
- Başarı Ölçütü: Border router başarıyla çalıştırıldı; IPv6 bağlantısı doğrulandı (Deney #432809).

Hafta 4 (Mart 2026, 1. Hafta): MQTT-SN Firmware Geliştirme

- emcute kütüphanesi tabanlı MQTT-SN ölçüm firmware'i (firmware/mqtt_node/main.c) geliştirildi. Firmware; broker bağlantısı, konu kaydı, 200 mesaj yayımı ve RTT hesaplama işlevlerini içermektedir.
- Zaman damgası mekanizması tasarlandı: ztimer_now(ZTIMER_MSEC) ile gönderme anı kaydedilmekte; broker üzerinden geri dönen echo'da alım anı ile fark alınmaktadır.
- Shell komutları (con, run) eklenerek interaktif test imkânı sağlandı.
- Başarı Ölçütü: Firmware derlemesi hatasız tamamlandı; yerel simülasyonda (native board) temel işlevsellik doğrulandı.

Hafta 5 (Mart 2026, 2. Hafta): İlk MQTT-SN Deneyleri

- Saclay sitesinde 5 düğüm ile ilk MQTT-SN deneyleri gerçekleştirildi (Deney #434055, 433620). Broker olarak Mosquitto RSMB yapılandırıldı (broker/rsmb.conf).
- Teknik Sorun: RSMB brokera bağlantı sağlanamadı (emcute_con failed). Analiz: Border router'ın IPv6 adresi düğüme eksik iletiliyordu. Çözüm: Con komutuna doğru broker IPv6 adresi açıkça girildi.
- Kısmi başarı elde edildi: 2 düğüm broker'a bağlanıp veri gönderdi; diğerleri zaman aşımına uğradı.
- Başarı Ölçütü: MQTT-SN bağlantısı 2 düğümde doğrulandı; sorun kök nedeni belirlendi.

Hafta 6 (Mart 2026, 3. Hafta): MQTT-SN Deneyi Başarı (Deney #434217)

- Deney #434217 başarıyla tamamlandı: Saclay, 10 A8-M3 düğüm (a8-18 ile a8-28), 120 dakika. Tüm düğümler broker'a bağlanarak veri gönderdi.
- Ağaç topolojisinde ölçülen RTT: ort. 15,95 ms, PDR: %100. Bu sonuç projenin en önemli başarısını temsil etmektedir.
- Deney verileri CSV formatında kaydedildi; yarımBASARILI5/ ve basarili5_tam/ klasörlerine arşivlendi.
- Başarı Ölçütü: MQTT-SN ağaç topolojisi deneyi %100 PDR ile tamamlandı; ölçüm verileri elde edildi.

Hafta 7 (Mart 2026, 4. Hafta): CoAP Firmware Geliştirme

- gcoap tabanlı CoAP ölçüm firmware'i (firmware/coap_node/main.c) geliştirildi. Rol tabanlı mimari benimsendi: bir düğüm sunucu, diğerleri istemci rolünde çalışmaktadır.
- /metrics/rtt CoAP kaynağı tanımlandı. Sunucu gelen POST yükünü echo olarak yanıtlamaktadır. İstemcide gcoap_req_send() ile yanıt callback mekanizması (mutex tabanlı senkronizasyon) uygulandı.
- Teknik Sorun: gcoap yanıt callback'inde race condition gözlemlendi. Çözüm: mutex_lock/unlock ile kritik bölge korundu.

- Başarı Ölçütü: CoAP firmware derlemesi tamamlandı; local simülasyonda echo mekanizması doğrulandı.

Hafta 8 (Nisan 2026, 1. Hafta): CoAP Yıldız Topolojisi Deneyi

- CoAP yıldız topolojisi deneyi gerçekleştirildi (Deney #434071). 31 başarılı paket çifti elde edildi; PDR: %100, ort. RTT: 17,67 ms.
- Wireshark/tcpdump ile PCAP yakalama yapılandırıldı (scripts/05_wireshark_capture.sh). CoAP paketleri Wireshark'ta doğrulandı (port 5683, COAP protokolü).
- CoAP Confirmable (CON) mesajlarının yeniden iletim mekanizmasının PDR'yi nasıl artırdığı gözlemlendi.
- Başarı Ölçütü: CoAP yıldız topolojisi %100 PDR ile tamamlandı.

Hafta 9 (Nisan 2026, 2. Hafta): CoAP Doğrusal ve Ağaç Topolojisi Deneyleri

- CoAP doğrusal ve ağaç topolojisi deneyleri gerçekleştirildi. TX power kısıtlaması uygulanarak çok atlamalı yönlendirme zorunlu hâle getirildi.
- Doğrusal topoloji: 13 çift, PDR %61,9, ort. RTT 17,97 ms. Ağaç topolojisi: 28 çift, PDR %84,85, ort. RTT 205,88 ms (aykırı değerler nedeniyle yüksek).
- Teknik Sorun: Ağaç topolojisinde bazı CoAP mesajları çok sayıda retransmission'a uğrayarak gecikmeyi 4,7 saniyeye çıkardı. Çözüm: Bu aykırı değerler analiz raporunda medyan değerle birlikte sunuldu.
- Başarı Ölçütü: Tüm CoAP topoloji deneyleri tamamlandı; PCAP dosyaları elde edildi.

Hafta 10 (Nisan 2026, 3. Hafta): MQTT-SN Yıldız ve Doğrusal Topoloji Deneyleri

- MQTT-SN yıldız ve doğrusal topoloji deneyleri gerçekleştirildi. Yıldız: 680 paket, PDR %38,46, ort. RTT 1983,90 ms. Doğrusal: 726 paket, yalnızca 2 başarılı çift.
- Yıldız topolojisindeki yüksek kayıp oranı araştırıldı: RSMB broker'ın senkron mesaj işleme modeli, çok sayıda düğümden eş zamanlı bağlantı isteği alındığında kuyruk taşmasına yol açmaktadır.
- Bu bulgu, MQTT-SN QoS 0 modunun onay mekanizması içermemesi nedeniyle yoğun trafik senaryolarında yetersiz kaldığını göstermektedir.
- Başarı Ölçütü: MQTT-SN yıldız ve doğrusal topoloji PCAP dosyaları elde edildi; kök neden analizi yapıldı.

Hafta 11 (Nisan 2026, 4. Hafta): Veri Analizi ve Görselleştirme

- Python tabanlı analiz araçları geliştirildi (analysis/ dizini). parse_metrics.py, PCAP dosyalarını Scapy kütüphanesiyle işleyerek CSV formatında çıktı üretmektedir. pcap_analyze.py ve pcap_deep.py ise derinlemesine istatistiksel analiz yapmaktadır.

- Karşılaştırmalı çubuk grafikler (comparison_plot.png, deep_comparison.png) Matplotlib kütüphanesiyle oluşturuldu.
- Wireshark ekran görüntüleri (wireshark_*.png) rapor kanıtı olarak arşivlendi.
- Başarı Ölçütü: Tüm PCAP dosyaları analiz edildi; görselleştirmeler tamamlandı.

Hafta 12 (Mayıs 2026, 1. Hafta): Otomasyon ve Tekrarlanabilirlik

- Deney yaşam döngüsü Makefile ve bash scriptleriyle otomatize edildi (scripts/ dizini, Makefile). Bu sayede gelecekte aynı deneyin farklı parametrelerle tekrarlanabilmesi sağlandı.
- 00_setup_tree_topology.sh, 01_submit.sh, 02_build.sh, 03_flash.sh, 04_run_mqtt.sh, 04_run_coap.sh, 05_wireshark_capture.sh scriptleri hazırlandı.
- Tüm deney kimlik numaraları ve yapılandırma parametreleri deneyler/ klasöründeki JSON dosyalarında (*.json) kayıt altına alındı.
- Başarı Ölçütü: Otomasyon scriptleri çalıştırıldı ve tekrarlanabilirlik doğrulandı.

Hafta 13 (Mayıs 2026, 2. Hafta): Rapor Hazırlama

- Vize raporu (hakandonmezvize.odt) ve IEEE formatında özet rapor (mqttsn_ieee_report.pdf) hazırlandı. Sonuçlar analiz edilerek tartışma bölümü oluşturuldu.
- Optimizasyon önerileri belgesi (optimizasyon_onerileri.docx) hazırlandı: QoS 1/2 kullanımı, adaptif yeniden iletim, topoloji-protokol eşleştirme rehberi gibi öneriler içermektedir.
- Sunum rehberi (sunum_rehberi_SANA_OZEL.docx) hazırlandı.
- Başarı Ölçütü: Vize raporu tamamlandı; tüm belgeler depo'ya yüklendi.

Hafta 14 (Mayıs 2026, 3–4. Hafta): Final Raporu ve Sunum Hazırlığı

- Final deney raporu hazırlandı (bu belge). Tüm deney bulguları akademik yazım standartlarına uygun biçimde derlendi; tablolar, şekiller ve kaynakça eklendi.
- Projenin özgün değeri netleştirildi: Gerçek donanım üzerinde, üç farklı topolojide, tekrarlanabilir deney altyapısıyla iki IoT protokolünün kapsamlı karşılaştırması.
- Depo son hâliyle güncellendi; GitHub linki proje formuna eklendi.
- Başarı Ölçütü: Final raporu tamamlandı; sunum hazır; tüm dosyalar UZEM'e yüklendi.

Tablo 7. 14 Haftalık İş Planı ve Başarı Ölçütleri

Hafta	Ana Faaliyet	Çıktı	Başarı Ölçütü
1	Literatür taraması, proje kurulumu	Proje formu, GitHub deposu	Form dolduruldu, depo açıldı
2	RIOT OS kurulumu, ilk rezervasyon	Derlenebilir ortam	Derleme hatasız tamamlandı
3	Border router yapılandırması	IPv6 bağlantısı	ping başarılı

4	MQTT-SN firmware geliştirme	mqtt_node/main.c	Native simülasyon başarılı
5	İlk MQTT-SN deneyleri	Kısmi bağlantı verisi	2 düğüm bağlandı
6	MQTT-SN ağaç topolojisi deneyi	Deney #434217 verisi	PDR=%100, RTT<20ms
7	CoAP firmware geliştirme	coap_node/main.c	Echo mekanizması doğrulandı
8	CoAP yıldız topolojisi deneyi	coap_star.pcap	PDR=%100
9	CoAP doğrusal+ağaç deneyleri	coap_linear/tree.pcap	PCAP dosyaları elde edildi
10	MQTT-SN yıldız+doğrusal deneyleri	mqttsn_star/linear.pcap	Kök neden analizi yapıldı
11	Veri analizi, görselleştirme	CSV, PNG grafikler	Grafikler tamamlandı
12	Otomasyon scriptleri	Makefile, bash scriptleri	Tekrarlanabilirlik doğrulandı
13	Vize raporu, IEEE özet	.odt, .pdf raporlar	Raporlar tamamlandı
14	Final raporu, sunum hazırlığı	Bu belge, sunum	UZEM'e yüklendi

8. SONUÇ VE ÖNERİLER

Bu çalışmada, FIT IoT-LAB test yatağında RIOT OS tabanlı A8-M3 düğümler üzerinde MQTT-SN ve CoAP protokollerinin başarımı; yıldız, doğrusal ve ağaç topolojilerinde deneysel olarak ölçülmüş ve karşılaştırılmıştır. Temel bulgular aşağıda özetlenmiştir:

- MQTT-SN, ağaç topolojisinde son derece düşük ve kararlı gecikme (15,95 ms RTT, %100 PDR) sergilemiştir. Bu bulgu, MQTT-SN'in RPL yönlendirme hiyerarşisiyle örtüştüğünü ve hiyerarşik IoT ağları için güçlü bir aday olduğunu ortaya koymaktadır.
- CoAP, yıldız topolojisinde %100 PDR elde ederek yüksek güvenilirlik göstermiştir. CON mesaj tipi, otomatik yeniden iletim sayesinde paket kayıplarını telafi etmektedir.
- Her iki protokol de topolojik mesafe arttıkça gecikme ve paket kayıplarının arttığını göstermiş; bu durum ağ tasarımının protokol başarımı üzerindeki belirleyici rolünü vurgulamaktadır.
- Tek bir protokolün tüm topolojilerde üstün olmadığı gözlemlenmiştir; uygulama gereksinimine göre protokol ve topoloji birlikte seçilmelidir.

Gelecek çalışmalar için öneriler şunlardır: (1) MQTT-SN QoS 1/2 modunun uygulanması ve PDR etkisinin incelenmesi, (2) DTLS güvenlik katmanının eklenmesiyle enerji ve gecikme maliyetinin ölçülmesi, (3) Düğüm sayısının artırılmasıyla ölçeklenebilirlik analizinin yapılması, (4) enerji tüketimi metriğinin (uA cinsinden akım ölçümü) deneysel süreçte dahil edilmesi.

Bu çalışma; gerçek donanım üzerinde tekrarlanabilir deney altyapısı kurarak elde ettiği ölçüm verileriyle, IoT protokolü seçim kılavuzlarına somut ve veri odaklı bir katkı sunmaktadır. Tüm deney verileri, firmware kaynak kodları ve analiz araçları GitHub deposunda kamuya açık biçimde paylaşılmıştır.

KAYNAKÇA

- [1] Atzori, L., Iera, A., & Morabito, G. (2010). The Internet of Things: A survey. *Computer Networks*, 54(15), 2787–2805. <https://doi.org/10.1016/j.comnet.2010.05.010>
- [2] Bormann, C., Castellani, A. P., & Shelby, Z. (2012). CoAP: An application protocol for billions of tiny Internet nodes. *IEEE Internet Computing*, 16(2), 62–67.
- [3] Stanford-Clark, A., & Nipper, H. (2013). MQTT for Sensor Networks (MQTT-SN) Protocol Specification Version 1.2. OASIS.
- [4] Shelby, Z., Hartke, K., & Bormann, C. (2014). The Constrained Application Protocol (CoAP). IETF RFC 7252.
- [5] Palmese, F., et al. (2018). CoAP vs. MQTT-SN: Comparison and Performance Evaluation in Publish-Subscribe Environments. *Proceedings WF-IoT 2018*.
- [6] Colitti, W., Steenhaut, K., & De Caro, N. (2011). Integrating wireless sensor networks with the web. *Proceedings of the Workshop on Extending the Internet to Low power and Lossy Networks (IP+SN)*.
- [7] Villaverde, B. C., et al. (2012). Service discovery protocols for constrained machine-to-machine communications. *IEEE Communications Surveys & Tutorials*, 16(1), 41–60.
- [8] Adjih, C., et al. (2015). FIT IoT-LAB: A large scale open experimental IoT testbed. *IEEE World Forum on Internet of Things (WF-IoT)*, 459–464.
- [9] Baccelli, E., et al. (2013). RIOT OS: Towards an OS for the Internet of Things. *Proceedings of the 32nd IEEE International Conference on Computer Communications (INFOCOM)*.
- [10] Montenegro, G., et al. (2007). Transmission of IPv6 packets over IEEE 802.15.4 networks. IETF RFC 4944.
- [11] Winter, T., et al. (2012). RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks. IETF RFC 6550.